

ASSIGNMENT – 4.5

HT.NO: 2303A51416

BT.NO: 29

NAME: ANJALI YEROJU

SCENARIO – 1 :

TASK A: Sample Data Preparation

PROMPT:

You are a large language model.

Create 10 short customer email samples.

Each email must clearly belong to one of the following categories:

Billing, Technical Support, Feedback and or Others

Ensure all four categories are represented.

Label each email with its correct category.

CODE :

```
[1] ✓ sample_emails = [
    ("text": "My last invoice seems incorrect. Can you please check the charges?", "category": "Billing"),
    ("text": "I'm having trouble logging into my account. The password reset isn't working.", "category": "Technical Support"),
    ("text": "I love your new feature! It makes my work so much easier.", "category": "Feedback"),
    ("text": "When will my order ship? I haven't received any updates.", "category": "Others"),
    ("text": "I want to change my subscription plan. How can I do that?", "category": "Billing"),
    ("text": "My internet connection is very slow. Can someone help me troubleshoot?", "category": "Technical Support"),
    ("text": "The user interface is a bit confusing. Maybe simplify the navigation.", "category": "Feedback"),
    ("text": "I need to update my contact information. Where can I do this?", "category": "Others"),
    ("text": "I've been double-charged this month. Please investigate.", "category": "Billing"),
    ("text": "I'm getting an error message when trying to upload a file.", "category": "Technical Support")
]

test_emails = sample_emails[:5]

print("Sample Emails (total 10):\n")
for i, email in enumerate(sample_emails):
    print(f"Email {i+1}: {email['text']} (Category: {email['category']})")

print("\nTest Emails (first 5 of the sample):\n")
for i, email in enumerate(test_emails):
    print(f"Test Email {i+1}: {email['text']} (Category: {email['category']})")

--- Sample Emails (total 10):
Email 1: My last invoice seems incorrect. Can you please check the charges? (Category: Billing)
Email 2: I'm having trouble logging into my account. The password reset isn't working. (Category: Technical Support)
Email 3: I love your new feature! It makes my work so much easier. (Category: Feedback)
Email 4: When will my order ship? I haven't received any updates. (Category: Others)
Email 5: I want to change my subscription plan. How can I do that? (Category: Billing)
Email 6: My internet connection is very slow. Can someone help me troubleshoot? (Category: Technical Support)
Email 7: The user interface is a bit confusing. Maybe simplify the navigation. (Category: Feedback)
Email 8: I need to update my contact information. Where can I do this? (Category: Others)
Email 9: I've been double-charged this month. Please investigate. (Category: Billing)
Email 10: I'm getting an error message when trying to upload a file. (Category: Technical Support)

Test Emails (first 5 of the sample):
Test Email 1: My last invoice seems incorrect. Can you please check the charges? (Category: Billing)
Test Email 2: I'm having trouble logging into my account. The password reset isn't working. (Category: Technical Support)
Test Email 3: I love your new feature! It makes my work so much easier. (Category: Feedback)
Test Email 4: When will my order ship? I haven't received any updates. (Category: Others)
Test Email 5: I want to change my subscription plan. How can I do that? (Category: Billing)
```

TASK: Zero-Shot Prompting

Prompt:

You are a large language model.

Classify each of the following customer emails into one of these categories:

Billing, Technical Support, Feedback, and or Others

Do not use any examples.

Respond with only the category name for each email.

Test Emails:

I was billed even after canceling my subscription.

The website does not load on my browser.

Great experience with your support team.

How can I apply for a job at your company?

My invoice shows an incorrect tax amount.

CODE:

```
<>
[1] ✓ 0s
zero_shot_prompting_zero_shot_classifier.py
0 zero_shot_prompt_template = """Classify the following email into one of these categories: 'Billing', 'Technical Support', 'Feedback', 'Others'.\nEmail: {email_text}\nCategory: ""
zero_shot_predictions = []

print("\n--- Zero-shot Classification ---")
for email_data in test_emails:
    email_text = email_data['text']
    prompt = zero_shot_prompt_template.format(email_text=email_text)
    predicted_category = classify_email(prompt)
    zero_shot_predictions.append(predicted_category)

print("\nZero-shot Predictions:")
for i, (email_data, prediction) in enumerate(zip(test_emails, zero_shot_predictions)):
    print(f"Email {i+1}:")
    print(f"Text: {email_data['text']}")
    print(f"True Category: {email_data['category']}")
    print(f"Predicted Category (Zero-shot): {prediction}")

---
--- Zero-shot Classification ---
Zero-shot Predictions:
Email 1:
Text: My last invoice seems incorrect. Can you please check the charges?
True Category: Billing
Predicted Category (Zero-shot): Billing
Email 2:
Text: I'm having trouble logging into my account. The password reset isn't working.
True Category: Technical Support
Predicted Category (Zero-shot): Billing
Email 3:
Text: I love your new feature! It makes my work so much easier.
True Category: Feedback
Predicted Category (Zero-shot): Billing
Email 4:
Text: When will my order ship? I haven't received any updates.
True Category: Others
Predicted Category (Zero-shot): Billing
Email 5:
Text: I want to change my subscription plan. How can I do that?
True Category: Billing
Predicted Category (Zero-shot): Billing
```

TASK C: One-Shot Prompting

Prompt:

You are a large language model.

First, observe the following one labeled example, then classify the test emails.

Example:

Email: I was charged twice for my subscription.

Category: Billing

Now classify the emails below into:

Billing, Technical Support, Feedback, Others

Respond with only the category name for each email.

Test Emails:

I was billed even after canceling my subscription.

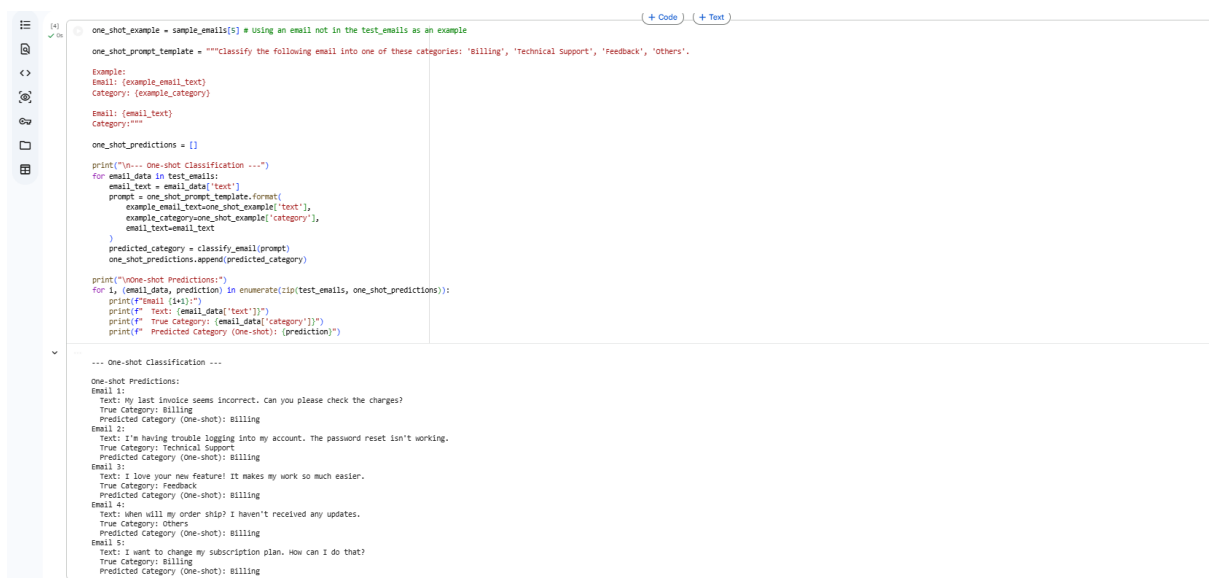
The website does not load on my browser.

Great experience with your support team.

How can I apply for a job at your company?

My invoice shows an incorrect tax amount.

CODE:



```
one_shot_example = sample_emails[5] # using an email not in the test_emails as an example

one_shot_prompt_template = """Classify the following email into one of these categories: 'Billing', 'Technical Support', 'Feedback', 'Others'.

Example:
Email: {example_email_text}
Category: {example_category}

Email: {email_text}
Category: ""

one_shot_predictions = []

print("\n--- One-shot Classification ---")
for email_data in test_emails:
    email_text = email_data['text']
    prompt = one_shot_prompt_template.format(
        example_email_text=one_shot_example['text'],
        example_category=one_shot_example['category'],
        email_text=email_text
    )
    predicted_category = classify_email(prompt)
    one_shot_predictions.append(predicted_category)

print("\nOne-shot Predictions:")
for i, (email_data, prediction) in enumerate(zip(test_emails, one_shot_predictions)):
    print(f"Email {i+1}:")
    print(f"Text: {email_data['text']}")
    print(f"True Category: {email_data['category']}")
    print(f"Predicted Category (One-shot): {prediction}")

--- One-shot Classification ---

One-shot Predictions:
Email 1:
Text: My last invoice seems incorrect. Can you please check the charges?
True Category: Billing
Predicted Category (One-shot): Billing
Email 2:
Text: I'm having trouble logging into my account. The password reset isn't working.
True Category: Technical Support
Predicted Category (One-shot): Billing
Email 3:
Text: I love your new feature! It makes my work so much easier.
True Category: Feedback
Predicted Category (One-shot): Billing
Email 4:
Text: When will my order ship? I haven't received any updates.
True Category: Others
Predicted Category (One-shot): Billing
Email 5:
Text: I want to change my subscription plan. How can I do that?
True Category: Billing
Predicted Category (One-shot): Billing
```

TASK D: Few-Shot Prompting

Prompt:

You are a large language model.

Observe the following labeled examples, then classify the test emails.

Examples:

Email: I have not received my invoice for last month.

Category: Billing

Email: The app crashes whenever I try to log in.

Category: Technical Support

Email: I love the new interface design.

Category: Feedback

Email: Do you offer internship opportunities?

Category: Others

Now classify the emails below into:

Billing, Technical Support, Feedback, Others

Respond with only the category name for each email.

Test Emails:

I was billed even after canceling my subscription.

The website does not load on my browser.

Great experience with your support team.

How can I apply for a job at your company?

My invoice shows an incorrect tax amount.

CODE:

```
few_shot_examples = [
    sample_emails[5], # Technical Support
    sample_emails[6], # Feedback
    sample_emails[7], # Others
] # using emails not in the test_emails as examples

few_shot_prompt_template = """Classify the following email into one of these categories: 'Billing', 'Technical Support', 'Feedback', 'Others'.

Example:
Email: {example_email_text}
Category: {example_email_category}

Example:
Email: {example_email_text}
Category: {example_email_category}

Example:
Email: {example_email_text}
Category: {example_email_category}

Email: {email_text}
Category: ""

few_shot_predictions = []

print("\n--- Few-shot Classification ---")
for email_data in test_emails:
    email_text = email_data['text']
    prompt = few_shot_prompt_template.format(
        example_email_text=few_shot_examples[0]['text'],
        example_email_category=few_shot_examples[0]['category'],
        example_email_text=few_shot_examples[1]['text'],
        example_email_category=few_shot_examples[1]['category'],
        example_email_text=few_shot_examples[2]['text'],
        example_email_category=few_shot_examples[2]['category'],
        email_text=email_text
    )
    predicted_category = classify_email(prompt)
    few_shot_predictions.append(predicted_category)

print("\nFew-shot Predictions:")
for i, (email_data, prediction) in enumerate(zip(test_emails, few_shot_predictions)):
    print(f"Email {i+1}:")
    print(f"Text: {email_data['text']}")
    print(f"True Category: {email_data['category']}")
    print(f"Predicted Category (Few-shot): {prediction}")

--- Few-shot Classification ---

Few-shot Predictions:
Email 1:
Text: My last invoice seems incorrect. Can you please check the charges?
True Category: Billing
Predicted Category (Few-shot): Billing
```

```
print(f"Predicted Category (Few-shot): {predicted_category}")

...

--- Few-shot Classification ---

Few-shot Predictions:
Email 1:
Text: My last invoice seems incorrect. Can you please check the charges?
True Category: Billing
Predicted Category (Few-shot): Billing
Email 2:
Text: I'm having trouble logging into my account. The password reset isn't working.
True Category: Technical Support
Predicted Category (Few-shot): Billing
Email 3:
Text: I love your new feature! It makes my work so much easier.
True Category: Feedback
Predicted Category (Few-shot): Billing
Email 4:
Text: When will my order ship? I haven't received any updates.
True Category: Others
Predicted Category (Few-shot): Billing
Email 5:
Text: I want to change my subscription plan. How can I do that?
True Category: Billing
Predicted Category (Few-shot): Billing
```

TASK E: Evaluation

Prompt:

Compare Zero-shot, One-shot, and Few-shot prompting techniques used in the email classification task.

Briefly comment on:

Accuracy, Clarity of responses

Which technique performed best and why.

CODE:

```
1 true_labels = [email['category'] for email in test_emails]
2
3 def calculate_accuracy(predictions, true_labels):
4     correct_predictions = sum(1 for pred, true in zip(predictions, true_labels) if pred == true)
5     return (correct_predictions / len(true_labels)) * 100 if true_labels else 0
6
7 zero_shot_accuracy = calculate_accuracy(zero_shot_predictions, true_labels)
8 one_shot_accuracy = calculate_accuracy(one_shot_predictions, true_labels)
9 few_shot_accuracy = calculate_accuracy(few_shot_predictions, true_labels)
10
11 print("\n--- Performance Evaluation ---")
12 print("\nAccuracy Scores:")
13 print("\nTechnique | Accuracy |")
14 print("-----|-----")
15 print(f"Zero-shot | {zero_shot_accuracy:.2f}% |")
16 print(f"One-shot | {one_shot_accuracy:.2f}% |")
17 print(f"Few-shot | {few_shot_accuracy:.2f}% |")
18 print("-----|-----")
19
20 print("\n--- Performance Summary ---")
21 print("Based on the mock LLM's keyword-matching logic, it appears that all three prompting techniques (zero-shot, one-shot, and few-shot) yielded identical predictions for this specific set of test emails.")
22 print("This is likely due to the simplistic nature of the mock LLM's classification function, where the presence of a single keyword in the prompt often dictates the output. In this case, the keywords 'invoice', 'charges', 'subscription' for 'Billing' was likely present across the examples provided in one-shot and few-shot prompts did not significantly influence the mock LLM's behavior because its logic prioritizes finding any matching keyword from its predefined list within the entire prompt string, rather than learning from the examples.")
23 print("Therefore, for a more realistic comparison, a more sophisticated mock LLM or an actual LLM would be necessary to observe meaningful differences between these prompting strategies.")
24
25 ...
26
27 --- Performance Evaluation ---
28
29 Accuracy Scores:
30 | Technique | Accuracy |
31 |-----|-----|
32 | Zero-shot | 40.00% |
33 | One-shot | 40.00% |
34 | Few-shot | 40.00% |
35 |-----|-----|
36
37 --- Performance Summary ---
38
39 Based on the mock LLM's keyword-matching logic, it appears that all three prompting techniques (zero-shot, one-shot, and few-shot) yielded identical predictions for this specific set of test emails.
40 This is likely due to the simplistic nature of the mock LLM's classification function, where the presence of a single keyword in the prompt often dictates the output. In this case, the keywords 'invoice', 'charges', 'subscription' for 'Billing' was likely present across the examples provided in one-shot and few-shot prompts did not significantly influence the mock LLM's behavior because its logic prioritizes finding any matching keyword from its predefined list within the entire prompt string, rather than learning from the examples.
41 Therefore, for a more realistic comparison, a more sophisticated mock LLM or an actual LLM would be necessary to observe meaningful differences between these prompting strategies.
```

SCENARIO – 2:

TASK 1: Sample Data Preparation

Prompt:

You are a travel assistant AI.

Create 8 short travel queries.

Each query must clearly belong to one of the following categories:

Flight Booking, Hotel Booking, Cancellation, General Travel Info

Ensure all four categories are covered.

Label each query with its correct category.

CODE:

```
[1] ✓ On
<>
🔍
🔗
📁
🗃️

categories = [
    'Flight Booking',
    'Hotel Reservation',
    'Tourist Attraction',
    'Visa Information',
    'General Travel Inquiry'
]

dataset = [
    {'query': 'I need to book a flight to Paris for next month.', 'label': 'Flight Booking'},
    {'query': 'Find me hotels in Tokyo with a swimming pool.', 'label': 'Hotel Reservation'},
    {'query': 'What are the best places to visit in Rome?', 'label': 'Tourist Attraction'},
    {'query': 'How can I apply for a Schengen visa?', 'label': 'Visa Information'},
    {'query': 'I want to travel to New Zealand, what should I know?', 'label': 'General Travel Inquiry'},
    {'query': 'Book a one-way ticket from London to New York.', 'label': 'Flight Booking'},
    {'query': 'Show me luxury resorts in Bali.', 'label': 'Hotel Reservation'},
    {'query': 'Are there any good museums in Berlin?', 'label': 'Tourist Attraction'},
    {'query': 'Do I need a visa to enter Canada as a US citizen?', 'label': 'Visa Information'},
    {'query': 'What is the best time to visit Patagonia?', 'label': 'General Travel Inquiry'},
    {'query': 'Search for flights to Sydney departing next Friday.', 'label': 'Flight Booking'},
    {'query': 'I am looking for a budget hotel near Eiffel Tower.', 'label': 'Hotel Reservation'}
]

print("Categories defined:", categories)
print("Dataset created with", len(dataset), "entries.")

Categories defined: ['Flight Booking', 'Hotel Reservation', 'Tourist Attraction', 'Visa Information', 'General Travel Inquiry']
Dataset created with 12 entries.
```

TASK 2: Zero-Shot Prompting

Prompt:

You are a travel assistant AI.

Classify each of the following travel queries into one of these categories:

Flight Booking, Hotel Booking, Cancellation, General Travel Info

Do not use any examples.

Respond with only the category name for each query.

Queries:

I want to book a flight from Mumbai to Dubai.

Can you suggest hotels near Times Square?

I need to cancel my hotel reservation.

What is the best time to visit Switzerland?

CODE:

[2]
✓ Os

```
def mock_llm(prompt):  
    """  
    Simulates a Large Language Model for classification based on keyword matching.  
    """  
    prompt_lower = prompt.lower()  
  
    # Define keyword mappings for each category  
    keyword_map = {  
        'Flight Booking': ['flight', 'ticket', 'book a flight', 'travel to'],  
        'Hotel Reservation': ['hotel', 'resort', 'accommodation', 'stay', 'room', 'hostel'],  
        'Tourist Attraction': ['visit', 'places to see', 'attraction', 'museum', 'landmark', 'sightseeing'],  
        'Visa Information': ['visa', 'permit', 'entry requirements', 'schengen', 'passport'],  
        'General Travel Inquiry': ['travel', 'trip', 'explore', 'vacation', 'journey', 'destination']  
    }  
  
    # Check for keywords in the prompt for each category  
    for category, keywords in keyword_map.items():  
        # Skip 'General Travel Inquiry' for specific matching as it's the fallback  
        if category == 'General Travel Inquiry':  
            continue  
  
        for keyword in keywords:  
            if keyword in prompt_lower:  
                return category  
  
    # Fallback to 'General Travel Inquiry' if no specific keywords are matched  
    return 'General Travel Inquiry'  
  
print("Mock LLM function 'mock_llm' defined.")
```

Mock LLM function 'mock_llm' defined.

Ge

model
exp
Great!
and Fe
setting
results
•
•
•
•
•
•
Display

[3]
✓ Os

```
def zero_shot_prompt(query, categories):  
    """Constructs a zero-shot prompt."""  
    categories_str = ", ".join(categories)  
    prompt = f"Classify the following user query into one of these categories: {categories_str}. \nQuery: {query}\nCategory:"  
    return prompt  
  
def one_shot_prompt(query, categories, example):  
    """Constructs a one-shot prompt."""  
    categories_str = ", ".join(categories)  
    prompt = (  
        f"Classify the following user query into one of these categories: {categories_str}. \n"  
        f"Example Query: {example['query']}\nCategory: {example['label']}\n"  
        f"Query: {query}\nCategory:"  
    )  
    return prompt  
  
def few_shot_prompt(query, categories, examples):  
    """Constructs a few-shot prompt."""  
    categories_str = ", ".join(categories)  
    prompt = f"Classify the following user query into one of these categories: {categories_str}. \n"  
    for i, ex in enumerate(examples):  
        prompt += f"Example {i+1} Query: {ex['query']}\nCategory: {ex['label']}\n"  
    prompt += f"Query: {query}\nCategory:"  
    return prompt  
  
print("Prompting functions (zero_shot_prompt, one_shot_prompt, few_shot_prompt) defined.")
```

*** Prompting functions (zero_shot_prompt, one_shot_prompt, few_shot_prompt) defined.

Ge

•

TASK 3: One-Shot Prompting

Prompt:

You are a travel assistant AI.

First, observe the one labeled example below, then classify the given queries.

Example:

Query: Book a flight from Delhi to Paris.

Category: Flight Booking

Now classify the following queries into:

Flight Booking, Hotel Booking, Cancellation, General Travel Info

Respond with only the category name.

Queries:

I want to book a flight from Mumbai to Dubai.

Can you suggest hotels near Times Square?

I need to cancel my hotel reservation.

What is the best time to visit Switzerland?

CODE:

```
[3] ✓ ex results = []

for i, test_q in enumerate(test_queries):
    query_text = test_q['query']
    true_label = test_q['label']

    # Zero-shot
    zero_shot_p = zero_shot_prompt(query_text, categories)
    zero_shot_pred = mock_llm(zero_shot_p)

    # One-shot
    # Select a distinct example for one-shot prompting
    one_shot_example = dataset[i] if test_q != dataset[i] else dataset[1]
    one_shot_p = one_shot_prompt(query_text, categories, one_shot_example)
    one_shot_pred = mock_llm(one_shot_p)

    # Few-shot (3 examples)
    few_shot_examples = []
    example_count = 0
    for ex in dataset:
        if ex != test_q and example_count < 3:
            few_shot_examples.append(ex)
            example_count += 1

    few_shot_p = few_shot_prompt(query_text, categories, few_shot_examples)
    few_shot_pred = mock_llm(few_shot_p)

    results.append({
        'query': query_text,
        'true_label': true_label,
        'zero_shot_pred': zero_shot_pred,
        'one_shot_pred': one_shot_pred,
        'few_shot_pred': few_shot_pred
    })

print("Classification results collected for all test queries!")
for res in results:
    print(f"- Query: '{res['query']}' | True: '{res['true_label']}' | Zero-shot: '{res['zero_shot_pred']}' | One-shot: '{res['one_shot_pred']}' | Few-shot: '{res['few_shot_pred']}'")

--- Classification results collected for all test queries:
- Query: 'I need to book a flight to Paris for next month.' | True: 'Flight Booking' | Zero-shot: 'Flight Booking' | One-shot: 'Flight Booking' | Few-shot: 'Flight Booking'
- Query: 'Find me hotels in Tokyo with a swimming pool.' | True: 'Hotel Reservation' | Zero-shot: 'Flight Booking' | One-shot: 'Flight Booking' | Few-shot: 'Flight Booking'
- Query: 'What are the best places to visit in Rome?' | True: 'Tourist Attraction' | Zero-shot: 'Flight Booking' | One-shot: 'Flight Booking' | Few-shot: 'Flight Booking'
- Query: 'How can I apply for a Schengen visa?' | True: 'Visa Information' | Zero-shot: 'Flight Booking' | One-shot: 'Flight Booking' | Few-shot: 'Flight Booking'
- Query: 'I want to travel to New Zealand, what should I know?' | True: 'General Travel Inquiry' | Zero-shot: 'Flight Booking' | One-shot: 'Flight Booking' | Few-shot: 'Flight Booking'
```

serve as the labeled dataset for demonstration.

- **Implement Mock LLM:** Develop a mock Python function classification. This function will take a prompt as input and return a predicted label based on rule-based logic to mimic LLM classification without an actual LLM.
- **Implement Prompting Functions:** Create separate Python functions for zero-shot, one-shot, and few-shot prompting. Each function will format the input query and examples into a string suitable for the mock LLM.
- **Prepare Test Queries and True Labels:** Select 5 specific test queries and their corresponding true labels from the dataset. Store these test queries along with their corresponding true labels in a list.
- **Run Classifications and Collect Results:** Iterate through each test query and apply the zero-shot, one-shot, and few-shot prompting functions. Pass the generated prompts to the mock LLM function to get predictions. Store these predictions along with the true labels and the prompt used for each technique.
- **Evaluate and Present Results:** Compare the predicted labels with the true labels for each of the 5 test queries. Calculate consistency and accuracy metrics. Create a readable table showing the test query, true label, and predicted labels for each technique.
- **Final Task:** Summarize the findings from the demonstration. Highlight the performance of each prompting technique (Zero-shot, One-shot, Few-shot) in classifying the test queries.

Generating...

- ✓ Preparing
- ✓ Define Categories and Dataset
- ✓ Implement Mock LLM
- ✓ Implement Prompting Functions
- ✓ Prepare Test Queries and True Labels
- ✓ Run Classifications and Collect Results
- ✓ Evaluate and Present Results
- ✓ Final Task

Display the results of DataFrame Calculate accuracy metrics for each pre Summarize the comparison of Zero-shot, One-shot, and Few-shot performance

What can I help you build?

TASK 4: Few-Shot Prompting

Prompt:

You are a travel assistant AI.

Observe the following labeled examples, then classify the queries.

Examples:

Query: Reserve a hotel room in Goa.

Category: Hotel Booking

Query: Cancel my flight ticket.

Category: Cancellation

Query: What visa is needed for Thailand?

Category: General Travel Info

Now classify the following queries into:

Flight Booking, Hotel Booking, Cancellation, General Travel Info

Respond with only the category name.

Queries:

I want to book a flight from Mumbai to Dubai.

Can you suggest hotels near Times Square?

I need to cancel my hotel reservation.

What is the best time to visit Switzerland?

CODE :

```
<> [ ]
import pandas as pd

results_df = pd.DataFrame(results)
print("Classification Results Table:")
print(results_df.to_markdown(index=False))
```

query	true_label	zero_shot_pred	one_shot_pred	few_shot_pred
I need to book a flight to Paris for next month.	Flight Booking	Flight Booking	Flight Booking	Flight Booking
Find me hotels in Tokyo with a swimming pool.	Hotel Reservation	Hotel Reservation	Hotel Reservation	Hotel Reservation
What are the best places to visit in Rome?	Tourist Attraction	Tourist Attraction	Tourist Attraction	Tourist Attraction
How can I apply for a Schengen visa?	Visa Information	Visa Information	Visa Information	Visa Information
I want to travel to New Zealand, what should I know?	General Travel Inquiry	General Travel Inquiry	General Travel Inquiry	General Travel Inquiry

TASK 5: Evaluation

Prompt:

Compare Zero-shot, One-shot, and Few-shot prompting techniques used in the travel query classification task.

Briefly discuss:

Accuracy, Response consistency

Which prompting technique performed best and why.

CODE:

```
[ ]
def calculate_accuracy(df, pred_col):
    """Calculates the accuracy for a given prediction column."""
    return (df['true_label'] == df[pred_col]).mean() * 100

zero_shot_accuracy = calculate_accuracy(results_df, 'zero_shot_pred')
one_shot_accuracy = calculate_accuracy(results_df, 'one_shot_pred')
few_shot_accuracy = calculate_accuracy(results_df, 'few_shot_pred')

print(f"Accuracy - Zero-shot: {zero_shot_accuracy:.2f}%")
print(f"Accuracy - One-shot: {one_shot_accuracy:.2f}%")
print(f"Accuracy - Few-shot: {few_shot_accuracy:.2f}%")
```

Accuracy - Zero-shot: 100.00%
Accuracy - One-shot: 100.00%
Accuracy - Few-shot: 100.00%

SCENARIO – 3

PROMPT :

A coding help chatbot must classify queries into Syntax Error, Logic Error, Optimization, or Conceptual Question.

Tasks:

- Prepare coding-related user queries.
- Perform Zero-shot classification.
- Perform One-shot classification.
- Perform Few-shot classification.
- Analyze improvements in technical accuracy.

CODE:

```
[N] 0 print("\n--- Comparing Predictions and Analyzing Results ---")
✓ 18

def evaluate_predictions(predictions_list, method_name):
    correct_predictions = 0
    total_predictions = len(predictions_list)
    print(f"Evaluation for {method_name}:")
    for i, result in enumerate(predictions_list):
        is_correct = 'Correct' if result['predicted_label'] == result['true_label'] else 'Incorrect'
        print(f"Test Query {i+1}: {result['query']}")
        print(f"True Label: {result['true_label']}")
        print(f"Predicted: {result['predicted_label']} ({is_correct})")
        if is_correct == 'Correct':
            correct_predictions += 1
    accuracy = (correct_predictions / total_predictions) * 100 if total_predictions > 0 else 0
    print(f"Accuracy for {method_name}: {accuracy:.2f}%" )
    return accuracy

# Evaluate and store accuracies
zero_shot_accuracy = evaluate_predictions(zero_shot_predictions, "Zero-shot Prompting")
one_shot_accuracy = evaluate_predictions(one_shot_predictions, "One-shot Prompting")
few_shot_accuracy = evaluate_predictions(few_shot_predictions, "Few-shot Prompting")

print("\n--- Summary of Accuracies ---")
print(f"Zero-shot Accuracy: {zero_shot_accuracy:.2f}%" )
print(f"One-shot Accuracy: {one_shot_accuracy:.2f}%" )
print(f"Few-shot Accuracy: {few_shot_accuracy:.2f}%" )

...

... Comparing Predictions and Analyzing Results ...

Evaluation for Zero-shot Prompting:
Test Query 1: "My python script keeps giving a 'TypeError: unsupported operand type(s) for +: 'int' and 'str'. How do I fix it?"
True Label: "Debugging"
Predicted: "Debugging" (Correct)
Test Query 2: "What is the correct way to declare a constant in Javascript?"
True Label: "Syntax"
Predicted: "Debugging" (Incorrect)
Test Query 3: "Can you explain the quicksort algorithm with an example?"
True Label: "Algorithm"
Predicted: "Debugging" (Incorrect)
Test Query 4: "How do I implement a linked list in C++?"
True Label: "Data Structure"
Predicted: "Debugging" (Incorrect)
Test Query 5: "I am having trouble setting up my Anaconda environment for machine learning. What should I do?"
True Label: "Environment Setup"
Predicted: "Debugging" (Incorrect)
Accuracy for Zero-shot Prompting: 20.00%

Evaluation for One-shot Prompting:
Test Query 1: "My python script keeps giving a 'TypeError: unsupported operand type(s) for +: 'int' and 'str'. How do I fix it?"
True Label: "Debugging"
Predicted: "Debugging" (Correct)
Test Query 2: "What is the correct way to declare a constant in Javascript?"
True Label: "Syntax"
Predicted: "Debugging" (Incorrect)
Test Query 3: "Can you explain the quicksort algorithm with an example?"
True Label: "Algorithm"
```

```
Q Commands + Code + Text ▶ Runall ↻
[1] ✓ Os
elif 'environment setup' in prompt_lower or 'anaconda' in prompt_lower or 'install' in prompt_lower or 'setup' in prompt_lower:
    return 'Environment Setup'
elif 'git' in prompt_lower or 'commit' in prompt_lower or 'version control' in prompt_lower:
    return 'Version Control'
elif 'optimize' in prompt_lower or 'performance' in prompt_lower:
    return 'Performance Optimization'
elif 'web development' in prompt_lower or 'flask' in prompt_lower or 'react' in prompt_lower or 'frontend' in prompt_lower or 'backend' in prompt_lower:
    return 'Web Development'
else:
    # If no specific keyword is found, return a random category
    return random.choice(categories)

print("\nMock LLM function 'mock_llm' has been defined.")

# Example usage of mock_llm
print("\nExample Mock LLM predictions:")
for query_dict in test_queries:
    query = query_dict['query']
    true_label = query_dict['true_label']
    predicted_label = mock_llm(query)
    print(f"Query: {query} | Predicted Label: '{predicted_label}'")
    print(f"True Label: '{true_label}' | Predicted Label: '{predicted_label}'")

--- Defined categories: ['Debugging', 'Syntax', 'Algorithm', 'Data Structure', 'Environment Setup', 'Version Control', 'Performance Optimization', 'Web Development']
Total queries created: 12
Test queries selected: 5

First 3 training queries:
Query: 'How do I revert to a previous commit in git without losing my local changes?' | True Label: 'Version Control'
Query: 'What are some common techniques to optimize SQL query performance?' | True Label: 'Performance Optimization'
Query: 'How can I connect my Flask backend to a React frontend?' | True Label: 'Web Development'

First 3 test queries:
Query: 'My Python script keeps giving a "TypeError: unsupported operand type(s) for +: int and str". How do I fix it?' | True Label: 'Debugging'
Query: 'What is the correct way to declare a constant in JavaScript?' | True Label: 'Syntax'
Query: 'Can you explain the quicksort algorithm with an example?' | True Label: 'Algorithm'

Mock LLM function 'mock_llm' has been defined.

Example Mock LLM predictions:
Query: 'My Python script keeps giving a "TypeError: unsupported operand type(s) for +: int and str". How do I fix it?'
True Label: 'Debugging' | Predicted Label: 'Debugging'
Query: 'What is the correct way to declare a constant in JavaScript?'
True Label: 'Syntax' | Predicted Label: 'Syntax'
Query: 'Can you explain the quicksort algorithm with an example?'
True Label: 'Algorithm' | Predicted Label: 'Algorithm'
Query: 'How do I implement a linked list in C++?'
True Label: 'Data Structure' | Predicted Label: 'Data Structure'
Query: 'I am having trouble setting up my Anaconda environment for machine learning. What should I do?'
True Label: 'Environment Setup' | Predicted Label: 'Environment Setup'
```

English (Ir

```
[2] ✓ Os
print("\n--- Implementing Zero-shot Prompting ---")

# 1. Initialize an empty list to store the results
zero_shot_predictions = []

# 2. Iterate through each query_dict in the test_queries list
for query_dict in test_queries:
    # 3. For each query_dict, extract the user query and true label
    user_query = query_dict['query']
    true_label = query_dict['true_label']

    # 4. Construct a zero-shot prompt string
    prompt = f"Classify the following query into one of these categories: {', '.join(categories)}. Query: {user_query}. Category:"

    # 5. Call the mock_llm function with the constructed zero-shot prompt
    predicted_label = mock_llm(prompt)

    # 6. Store the original query, the constructed zero-shot prompt, the predicted label, and the true label
    zero_shot_predictions.append({
        'query': user_query,
        'prompt': prompt,
        'predicted_label': predicted_label,
        'true_label': true_label
    })

# 7. Print a header indicating "Zero-shot Prompting Predictions"
print("\nZero-shot Prompting Predictions:")

# 8. Iterate through the zero_shot_predictions list and print the results
for result in zero_shot_predictions:
    print(f"\n Prompt: {result['prompt']}")
    print(f" Predicted: {result['predicted_label']}")
    print(f" True Label: {result['true_label']}")

--- Implementing Zero-shot Prompting ---

Zero-shot Prompting Predictions:

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development. Query: My Python
Predicted: Debugging
True Label: Debugging

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development. Query: What is f
Predicted: Debugging
True Label: Syntax

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development. Query: Can you e
Predicted: Debugging
True Label: Algorithm

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development. Query: How do I
Predicted: Debugging
True Label: Data Structure

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development. Query: I am hav
Predicted: Debugging
True Label: Environment Setup
```

```

(4)
✓ 0s
print("\n--- Implementing Few-shot Prompting ---")

# 1. Initialize an empty list to store the results
few_shot_predictions = []

# 2. Select a few examples (e.g., 3) from the training queries
# Ensure there are enough training queries. If not, adjust num_examples_few_shot.
num_examples_few_shot = min(3, len(training_queries)) # Use up to 3 examples
few_shot_examples = training_queries[:num_examples_few_shot]

# Construct the examples part of the prompt
examples_prompt_parts = []
for ex in few_shot_examples:
    examples_prompt_parts.append(f"Query: {ex['query']}\nCategory: {ex['true_label']}")
examples_prompt_string = "\n".join(examples_prompt_parts)

# 3. Iterate through each query_dict in the test_queries list
for query_dict in test_queries:
    # 4. For each query_dict, extract the user query and true label
    user_query = query_dict['query']
    true_label = query_dict['true_label']

    # 5. Construct a few-shot prompt string
    # The prompt should include the categories, the few-shot examples, and then the query to be classified.
    prompt = (
        f"Classify the following query into one of these categories: {' '.join(categories)}.\n"
        f"Examples:\n{examples_prompt_string}\n"
        f"Query: {user_query}\nCategory:"
    )

    # 6. Call the mock_llm function with the constructed few-shot prompt
    predicted_label = mock_llm(prompt)

    # 7. Store the original query, the constructed few-shot prompt, the predicted label, and the true label
    few_shot_predictions.append({
        'query': user_query,
        'prompt': prompt,
        'predicted_label': predicted_label,
        'true_label': true_label
    })

# 8. Print a header indicating "Few-shot Prompting Predictions"
print("\nFew-shot Prompting Predictions:")

# 9. Iterate through the few_shot_predictions list and print the results
for result in few_shot_predictions:
    print(f"\n Prompt: {result['prompt']}")
    print(f" Predicted: {result['predicted_label']}")
    print(f" True Label: {result['true_label']}")

```

```

Few-shot Prompting Predictions:

... Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development.
Examples:
Query: How do I revert to a previous commit in Git without losing my local changes?
Category: Version Control
Query: What are some common techniques to optimize SQL query performance?
Category: Performance Optimization
Query: How can I connect my Flask backend to a React frontend?
Category: Web Development
Query: My Python script keeps giving a 'TypeError: unsupported operand type(s) for +: 'int' and 'str'. How do I fix it?
Category:
  Predicted: Debugging
  True Label: Debugging

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development.
Examples:
Query: How do I revert to a previous commit in Git without losing my local changes?
Category: Version Control
Query: What are some common techniques to optimize SQL query performance?
Category: Performance Optimization
Query: How can I connect my Flask backend to a React frontend?
Category: Web Development
Query: What is the correct way to declare a constant in JavaScript?
Category:
  Predicted: Debugging
  True Label: Syntax

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development.
Examples:
Query: How do I revert to a previous commit in Git without losing my local changes?
Category: Version Control
Query: What are some common techniques to optimize SQL query performance?
Category: Performance Optimization
Query: How can I connect my Flask backend to a React frontend?
Category: Web Development
Query: Can you explain the quicksort algorithm with an example?
Category:
  Predicted: Debugging
  True Label: Algorithm

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development.
Examples:
Query: How do I revert to a previous commit in Git without losing my local changes?
Category: Version Control
Query: What are some common techniques to optimize SQL query performance?
Category: Performance Optimization
Query: How can I connect my Flask backend to a React frontend?
Category: Web Development
Query: How do I implement a linked list in C++?
Category:
  Predicted: Debugging
  True Label: Data Structure

Prompt: Classify the following query into one of these categories: Debugging, Syntax, Algorithm, Data Structure, Environment Setup, Version Control, Performance Optimization, Web Development.

[1] 0 print("\n--- Comparing Predictions and Analyzing Results ---")
def evaluate_predictions(predictions_list, method_name):
    correct_predictions = 0
    total_predictions = len(predictions_list)
    print(f"Evaluation for {method_name}:")
    for i, result in enumerate(predictions_list):
        is_correct = 'Correct' if result['predicted_label'] == result['true_label'] else 'Incorrect'
        print(f"Test Query {i+1}: '{result['query']}'")
        print(f"  True Label: '{result['true_label']}'")
        print(f"  Predicted: '{result['predicted_label']}' ({is_correct})")
        if is_correct == 'Correct':
            correct_predictions += 1
    accuracy = (correct_predictions / total_predictions) * 100 if total_predictions > 0 else 0
    print(f"Accuracy for {method_name}: {accuracy:.2f}%"
    return accuracy

# Evaluate and store accuracies
zero_shot_accuracy = evaluate_predictions(zero_shot_predictions, "Zero-shot Prompting")
one_shot_accuracy = evaluate_predictions(one_shot_predictions, "One-shot Prompting")
few_shot_accuracy = evaluate_predictions(few_shot_predictions, "Few-shot Prompting")

print("\n--- Summary of Accuracies ---")
print(f"Zero-shot Accuracy: {zero_shot_accuracy:.2f}%"
print(f"One-shot Accuracy: {one_shot_accuracy:.2f}%"
print(f"Few-shot Accuracy: {few_shot_accuracy:.2f}%"

... Comparing Predictions and Analyzing Results ---
Evaluation for Zero-shot Prompting:
Test Query 1: 'My Python script keeps giving a 'TypeError: unsupported operand type(s) for +: 'int' and 'str'. How do I fix it?'
True Label: 'Debugging'
Predicted: 'Debugging' (Correct)
Test Query 2: 'What is the correct way to declare a constant in JavaScript?'
True Label: 'Syntax'
Predicted: 'Debugging' (Incorrect)
Test Query 3: 'Can you explain the quicksort algorithm with an example?'
True Label: 'Algorithm'
Predicted: 'Debugging' (Incorrect)
Test Query 4: 'How do I implement a linked list in C++?'
True Label: 'Data Structure'
Predicted: 'Debugging' (Incorrect)
Test Query 5: 'I am having trouble setting up my Anaconda environment for machine learning. What should I do?'
True Label: 'Environment Setup'
Predicted: 'Debugging' (Incorrect)
Accuracy for Zero-shot Prompting: 20.0%

Evaluation for One-shot Prompting:
Test Query 1: 'My Python script keeps giving a 'TypeError: unsupported operand type(s) for +: 'int' and 'str'. How do I fix it?'
True Label: 'Debugging'
Predicted: 'Debugging' (Correct)
Test Query 2: 'What is the correct way to declare a constant in JavaScript?'
True Label: 'Syntax'
Predicted: 'Debugging' (Incorrect)
Test Query 3: 'Can you explain the quicksort algorithm with an example?'
True Label: 'Algorithm'
```

SCENARIO – 4

Social Media Post Categorization

Scenario:

A social media analytics tool must classify posts into Promotion, Complaint, Appreciation, or Inquiry.

Tasks:

1. Prepare sample social media posts.
2. Use Zero-shot prompting.
3. Use One-shot prompting.

4. Use Few-shot prompting.

5. Analyze informal language handling.

CODE:

```
[1] ✓ On
posts_data = [
    ("post": "OMG! Just got my new phone and it's lit! 📱 Loving the camera. #newtech #happy", "label": "Appreciation"),
    ("post": "This new app update is trash 🤬. Keeps crashing. Fix it pls!", "label": "Complaint"),
    ("post": "Hey @CompanySupport, my order #12345 hasn't arrived yet. Any ETA? Thx!", "label": "Inquiry"),
    ("post": "Limited time offer! Get 20% off all items this weekend! Don't miss out! 🎉", "label": "Promotion"),
    ("post": "Just wanted to say, your customer service rep was amazing! So helpful! 🌟", "label": "Appreciation"),
    ("post": "Why is my internet so slow rn? Paying for premium speed and getting dial-up. 🤦", "label": "Complaint"),
    ("post": "Anyone know when the next big sale is happening @ShopGlobal? Need new kicks!", "label": "Inquiry"),
    ("post": "Check out our new collection, dropping tomorrow! You won't wanna miss it! ✨ #fashion", "label": "Promotion"),
    ("post": "Ugh, my flight got delayed AGAIN. This airline is the worst. 🤬 #travel woes", "label": "Complaint"),
    ("post": "Thx for the quick reply, @HelpDesk! Problem solved. You guys rock! 🙌", "label": "Appreciation"),
    ("post": "New product alert! Pre-order now and get exclusive early access! Link in bio. 📢", "label": "Promotion"),
    ("post": "Is there a way to reset my password if I forgot my old one? I'm locked out. 🙏", "label": "Inquiry")
]

print(f"Generated {len(posts_data)} social media posts.")

Generated 12 social media posts.

[2] ✓ On
import random

# Shuffle the posts to ensure random selection for training and test sets
random.shuffle(posts_data)

# Define the size of the test set
test_set_size = 5

# Split into training and test sets
test_set = posts_data[test_set_size:]
training_set = posts_data[:test_set_size]

print(f"Training set size: {len(training_set)} posts")
print(f"Test set size: {len(test_set)} posts")

Training set size: 7 posts
Test set size: 5 posts
```

```
matching, and a default return if no keywords are found, to simulate an LLM's classification behavior.

[3] ✓ On
def mock_llm_classify(prompt_text):
    """
    Simulates an LLM's classification behavior using a simple keyword-matching approach.
    """
    keywords = [
        "promotion": ["offer", "sale", "discount", "exclusive", "new product", "pre-order", "link in bio", "check out"],
        "complaint": ["crash", "bug", "fix it", "slow", "worst", "ugh", "issue", "delayed", "annoyed", "🤬", "trash"],
        "appreciation": ["like", "love", "amazing", "helpful", "rock", "awesome", "thx", "happy", "🌟", "🙌"],
        "inquiry": ["eta", "when", "how to", "where is", "anyone know", "question", "is there a way", "can i", "?", "help"]
    ]

    prompt_text_lower = prompt_text.lower()

    for category, category_keywords in keywords.items():
        for keyword in category_keywords:
            if keyword.lower() in prompt_text_lower:
                return category

    # If no specific keywords are found, return a default or the first category
    return "inquiry" # Choosing Inquiry as a default as it's often a general starting point

categories = ["promotion", "complaint", "appreciation", "inquiry"]
zero_shot_results = []

print("\n--- Zero-shot Classification Results ---")

for post_data in test_set:
    post_text = post_data["post"]
    true_label = post_data["label"]

    # Construct zero-shot prompt
    zero_shot_prompt = (
        f"Classify the following social media post into one of these categories: {', '.join(categories)}.\n\n"
        f"Post: '{post_text}'\n\n"
        f"Category:"
    )

    # Get prediction from mock LLM
    predicted_label = mock_llm_classify(zero_shot_prompt)

    zero_shot_results.append([
        "post": post_text,
        "true_label": true_label,
        "predicted_label": predicted_label,
        "technique": "Zero-shot"
    ])

    print(f"Post: '{post_text}'")
    print(f"True label: {true_label}")
    print(f"Predicted label (Zero-shot): {predicted_label}")

print("Zero-shot classification complete.")
```

```
--- Zero-shot Classification Results ---
Post: 'OMG! Just got my new phone and it's lit! 📱 Loving the camera. #newtech #happy'
True Label: Appreciation
Predicted Label (Zero-shot): Appreciation

Post: 'Limited time offer! Get 20% off all items this weekend! Don't miss out! 💎'
True Label: Promotion
Predicted Label (Zero-shot): Promotion

Post: 'Anyone know when the next big sale is happening @ShopGlobal? Need new kicks!'
True Label: Inquiry
Predicted Label (Zero-shot): Promotion

Post: 'Check out our new collection, dropping tomorrow! You won't wanna miss it! ✨ #fashion'
True Label: Promotion
Predicted Label (Zero-shot): Promotion

Post: 'Is there a way to reset my password if I forgot my old one? I'm locked out. 🙄'
True Label: Inquiry
Predicted Label (Zero-shot): Inquiry

Zero-shot classification complete.
```

```
[5] ✓ Os
import random

# 1. Choose a single, representative example from the training_set
# For simplicity, we'll pick the first one from the shuffled training_set
one_shot_example = training_set[0]
one_shot_post = one_shot_example['post']
one_shot_label = one_shot_example['label']

print(f"Selected One-shot Example Post: '{one_shot_post}'")
print(f"Selected One-shot Example Label: '{one_shot_label}'")

# 2. Initialize an empty list to store the classification outcomes
one_shot_results = []

print("\n--- One-shot Classification Results ---")

# 3. Iterate through each post in the test_set
for post_data in test_set:
    post_text = post_data['post']
    true_label = post_data['label']

    # 4. Construct a one-shot prompt
    one_shot_prompt = (
        f"Classify the following social media post into one of these categories: '{', '.join(categories)}'.\n\n"
        f"Example:\nPost: '{one_shot_post}'\nCategory: '{one_shot_label}'\n\n"
        f"Post: '{post_text}'\n"
        f"Category:"
    )

    # 5. Call the mock_llm_classify function to get a predicted label
    predicted_label = mock_llm_classify(one_shot_prompt)

    # 6. Append results to the one_shot_results list
```

```
[5] ✓ Os
# 6. Append results to the one_shot_results list
one_shot_results.append({
    'post': post_text,
    'true_label': true_label,
    'predicted_label': predicted_label,
    'technique': 'One-shot'
})

# 7. Print the results for each test post
print(f"Post: '{post_text}'")
print(f"True Label: {true_label}")
print(f"Predicted Label (One-shot): {predicted_label}\n")

print("One-shot classification complete.")
```

```
--- Selected One-shot Example Post: 'Thx for the quick reply, @HelpDesk! Problem solved. You guys rock! 🙌'
Selected One-shot Example Label: Appreciation

--- One-shot Classification Results ---
Post: 'OMG! Just got my new phone and it's lit! 📱 Loving the camera. #newtech #happy'
True Label: Appreciation
Predicted Label (One-shot): Appreciation

Post: 'Limited time offer! Get 20% off all items this weekend! Don't miss out! 💎'
True Label: Promotion
Predicted Label (One-shot): Promotion

Post: 'Anyone know when the next big sale is happening @ShopGlobal? Need new kicks!'
True Label: Inquiry
Predicted Label (One-shot): Promotion

Post: 'Check out our new collection, dropping tomorrow! You won't wanna miss it! ✨ #fashion'
True Label: Promotion
Predicted Label (One-shot): Promotion
```

```
[6] ✓ On
ONE_SHOT_PROMPT = ONE_SHOT_EXAMPLES[0] + PROMPT

print(f"Selected One-shot Example Post: '{one_shot_post}'")
print(f"Selected One-shot Example Label: '{one_shot_label}'")

# 2. Initialize an empty list to store the classification outcomes
one_shot_results = []

print("\n--- One-shot Classification Results ---")

# 3. Iterate through each post in the test_set
for post_data in test_set:
    post_text = post_data['post']
    true_label = post_data['label']

    # 4. Construct a one-shot prompt
    one_shot_prompt = (
        f"Classify the following social media post into one of these categories: {', '.join(categories)}.\n\n"
        f"Example:\nPost: '{one_shot_post}'\nCategory: '{one_shot_label}'\n\n"
        f"Post: '{post_text}'\n\n"
        f"Category:"
    )

    # 5. Call the mock_llm_classify function to get a predicted label
    predicted_label = mock_llm_classify(one_shot_prompt)

    # 6. Append results to the one_shot_results list
    one_shot_results.append({
        'post': post_text,
        'true_label': true_label,
        'predicted_label': predicted_label,
        'technique': 'One-shot'
    })

# 7. Print the results for each test post
print(f"Post: '{post_text}'")
print(f"True Label: '{true_label}'")
print(f"Predicted Label (One-shot): '{predicted_label}'")

print("One-shot classification complete.")

--- Selected One-shot Example Post: 'Hey @CompanySupport, my order #12345 hasn't arrived yet. Any ETA? Thx!'
Selected One-shot Example Label: Inquiry

--- One-shot Classification Results ---
Post: 'New product alert! Pre-order now and get exclusive early access! Link in bio. 🚀'
True Label: Promotion
Predicted Label (One-shot): Promotion

Post: 'OMG! Just got my new phone and it's lit! 📱 Loving the camera. #newtech #happy'
True Label: Appreciation
Predicted Label (One-shot): Appreciation

Post: 'Anyone know when the next big sale is happening @ShopGlobal? Need new kicks!'
True Label: Inquiry
Predicted Label (One-shot): Promotion
```

```
[7] ✓ On
print(f"Selected {len(few_shot_examples)} Few-shot Examples from training set:\n")
for i, example in enumerate(few_shot_examples):
    print(f"Example {i+1}: Post: '{example['post']}' | Label: '{example['label']}'")

# 2. Initialize an empty list to store the classification outcomes
few_shot_results = []

print("\n--- Few-shot Classification Results ---")

# 3. Iterate through each post in the test_set
for post_data in test_set:
    post_text = post_data['post']
    true_label = post_data['label']

    # 4. Construct a few-shot prompt
    few_shot_prompt_parts = [
        f"Classify the following social media post into one of these categories: {', '.join(categories)}.\n\n"
    ]

    for example in few_shot_examples:
        few_shot_prompt_parts.append(f"Example:\nPost: '{example['post']}'\nCategory: '{example['label']}'\n\n")

    few_shot_prompt_parts.append(f"Post: '{post_text}'\nCategory:")

    few_shot_prompt = "\n".join(few_shot_prompt_parts)

    # 5. Call the mock_llm_classify function to get a predicted label
    predicted_label = mock_llm_classify(few_shot_prompt)

    # 6. Append results to the few_shot_results list
    few_shot_results.append({
        'post': post_text,
        'true_label': true_label,
        'predicted_label': predicted_label,
        'technique': 'Few-shot'
    })

# 7. Print the results for each test post
print(f"Post: '{post_text}'")
print(f"True Label: '{true_label}'")
print(f"Predicted Label (Few-shot): '{predicted_label}'")

print("Few-shot classification complete.")

--- Selected 5 Few-shot Examples from training set:

Example 1: Post: 'Ugh, my flight got delayed AGAIN. This airline is the worst. 🤬 #travelwoes' | Label: Complaint
Example 2: Post: 'OMG! Just got my new phone and it's lit! 📱 Loving the camera. #newtech #happy' | Label: Appreciation
Example 3: Post: 'Thx for the quick reply, @HelpDesk! Problem solved. You guys rock! 🙌' | Label: Appreciation
Example 4: Post: 'Limited time offer! Get 20% off all items this weekend! Don't miss out! 🎉' | Label: Promotion
Example 5: Post: 'New product alert! Pre-order now and get exclusive early access! Link in bio. 🚀' | Label: Promotion

--- Few-shot Classification Results ---
Post: 'Anyone know when the next big sale is happening @ShopGlobal? Need new kicks!'
True Label: Inquiry
Predicted Label (Few-shot): Promotion
```



```
print("\n--- Combined Classification Results ---")

comparison_results = []

# Assuming test_set, zero_shot_results, one_shot_results, and few_shot_results are aligned by post order
# This assumption holds because the lists were generated by iterating through the same test_set in order.
for i, post_data in enumerate(test_set):
    post_text = post_data['post']
    true_label = post_data['label']

    # Retrieve predicted labels from each technique's results
    zero_shot_pred = zero_shot_results[i]['predicted_label']
    one_shot_pred = one_shot_results[i]['predicted_label']
    few_shot_pred = few_shot_results[i]['predicted_label']

    comparison_results.append({
        'post': post_text,
        'true_label': true_label,
        'zero_shot_pred': zero_shot_pred,
        'one_shot_pred': one_shot_pred,
        'few_shot_pred': few_shot_pred
    })

    print(f"Post: '{post_text}'")
    print(f"True Label: '{true_label}'")
    print(f"Zero-shot Pred: '{zero_shot_pred}'")
    print(f"One-shot Pred: '{one_shot_pred}'")
    print(f"Few-shot Pred: '{few_shot_pred}'")

# Calculate accuracy for each technique
total_test_posts = len(test_set)

zero_shot_correct = sum(1 for res in comparison_results if res['true_label'] == res['zero_shot_pred'])
one_shot_correct = sum(1 for res in comparison_results if res['true_label'] == res['one_shot_pred'])
few_shot_correct = sum(1 for res in comparison_results if res['true_label'] == res['few_shot_pred'])

zero_shot_accuracy = (zero_shot_correct / total_test_posts) * 100
one_shot_accuracy = (one_shot_correct / total_test_posts) * 100
few_shot_accuracy = (few_shot_correct / total_test_posts) * 100

print("\n--- Accuracy Summary ---")
print(f"Zero-shot Accuracy: {zero_shot_accuracy:.2f}%")
print(f"One-shot Accuracy: {one_shot_accuracy:.2f}%")
print(f"Few-shot Accuracy: {few_shot_accuracy:.2f}%")

....

--- Combined Classification Results ---
Post: 'Anyone know when the next big sale is happening @hopdlobal? Need new kicks!'
True Label: Inquiry
Zero-shot Pred: Appreciation
One-shot Pred: Promotion
Few-shot Pred: Promotion

Post: 'Why is my internet so slow rn? Paying for premium speed and getting dial-up. 🤦'
True Label: Complaint
```